

Instructions

6 exercises — sync/async se lekar event loop tracing tak.

★ marked = interview mein frequently pooche jaate hain — carefully solve karo.

PREDICT questions mein mentally execute karo pehle — phir browser/Node mein verify karo.

Exercise 1 ★ — Event Loop Output Prediction (Classic Interview)

Concept: Event Loop / Queue Priority **Task:** Ye ek all-in-one event loop question hai — EXACT output order predict karo with explanation:

```
console.log('1');
setTimeout(() => console.log('2'), 0);
Promise.resolve()
  .then(() => {
    console.log('3');
    setTimeout(() => console.log('4'), 0);
  })
  .then(() => console.log('5'));
queueMicrotask(() => console.log('6'));
setTimeout(() => {
  console.log('7');
  Promise.resolve().then(() => console.log('8'));
}, 0);
console.log('9');

// Write the EXACT output order: ?, ?, ?, ?, ?, ?, ?, ?, ?
// Explain WHY each number appears in that position.
```

Key Insight / Answer

Order: 1 → 9 → 3 → 6 → 5 → 2 → 7 → 8 → 4

Sync first (1,9), then all microtasks (3 fires which adds 4-setTimeout, then 6, then 5).

Then macrotasks one at a time: 2 (first setTimeout), then 7 (second setTimeout — drains microtask 8 before 4).

4 is last because it was added AFTER the first macrotask (inside setTimeout of Promise.then).

Exercise 2 ★ — Call Stack Overflow + Fix

Concept: [Call Stack](#) **Task:** Ye code stack overflow deta hai. Kyun hota hai aur 2 alag ways se fix karo:

```
// ■ Stack Overflow
function processItems(items, index = 0) {
  if (index >= items.length) return;
  heavyProcess(items[index]);
  processItems(items, index + 1); // recursive – stack grows!
}
const bigArray = new Array(100000).fill(0).map((_, i) => i);
processItems(bigArray);
// RangeError: Maximum call stack size exceeded

// Fix 1: Iterative (convert recursion to loop)
function processItemsIterative(items) {
  for (let i = 0; i < items.length; i++) {
    heavyProcess(items[i]);
  }
}

// Fix 2: Async recursion using setTimeout (yields control back)
function processItemsAsync(items, index = 0) {
  if (index >= items.length) return;
  heavyProcess(items[index]);
  setTimeout(() => processItemsAsync(items, index + 1), 0);
  // Each recursive call goes through event loop – stack never grows deep!
}
processItemsAsync(bigArray);
// No stack overflow – but slower (each step is async)
```

Key Insight / Answer

Stack overflow: recursive call adds new frame every time — 100000 frames = crash.

Fix 1 (iterative): constant stack depth — only 1 frame at a time.

Fix 2 (setTimeout 0): each recursion yields to event loop — stack pops before next call.

Fix 2 tradeoff: async, so execution is spread over many event loop ticks.

Exercise 3 — Callback Hell: Refactor to Named Functions

Concept: [Callback Pattern](#) **Task:** Ye deeply nested callback hell code hai. Named functions technique se flatten karo (no Promises yet!):

```
// ■ Callback Hell — 4 levels deep
function loadUserDashboard(userId) {
  getUser(userId, (err, user) => {
    if (err) { showError(err); return; }
    getPosts(user.id, (err, posts) => {
      if (err) { showError(err); return; }
      getComments(posts[0].id, (err, comments) => {
        if (err) { showError(err); return; }
        getAuthor(comments[0].userId, (err, author) => {
          if (err) { showError(err); return; }
          renderDashboard(user, posts, comments, author);
        });
      });
    });
  });
}

// ■ REFACTORED — Named functions, flat structure
// Complete the refactoring below:
function handleAuthor(user, posts, comments) {
  return function(err, author) {
    if (err) { showError(err); return; }
    renderDashboard(user, posts, comments, author);
  };
}

function handleComments(user, posts) {
  return function(err, comments) {
    if (err) { showError(err); return; }
    getAuthor(comments[0].userId, handleAuthor(user, posts, comments));
  };
}

function handlePosts(user) {
  return function(err, posts) {
    if (err) { showError(err); return; }
    getComments(posts[0].id, handleComments(user, posts));
  };
}

function handleUser(err, user) {
  if (err) { showError(err); return; }
  getPosts(user.id, handlePosts(user));
}

function loadUserDashboard(userId) {
  getUser(userId, handleUser); // clean single line!
}
```

Key Insight / Answer

Factory functions (handleAuthor, handleComments) — closures jo outer data carry karte hain.

Har step sirf ek cheez karta hai — single responsibility.

Error handling still repeated but more manageable — ek handleError function bhi bana sakte ho.

Pyramid completely gone — flat, readable.

Exercise 4 ★ — setTimeout Timing Traps

Concept: [setTimeout Internals](#) **Task:** Har scenario ka output aur timing explain karo:

```
// Trap 1: setTimeout in a loop with var
for (var i = 1; i <= 3; i++) {
  setTimeout(() => console.log('var i:', i), i * 100);
}
// Output: ?

// Fix with let
for (let i = 1; i <= 3; i++) {
  setTimeout(() => console.log('let i:', i), i * 100);
}
// Output: ?

// Trap 2: setTimeout inside synchronous while loop
const start = Date.now();
setTimeout(() => console.log('timer fired after:', Date.now()-start, 'ms'), 100);
while (Date.now() - start < 500) {} // blocks for 500ms!
console.log('while done');
// Does timer fire at 100ms? Explain.

// Trap 3: Nested setTimeouts timing
setTimeout(() => {
  console.log('outer', Date.now() - start);
  setTimeout(() => {
    console.log('inner', Date.now() - start);
  }, 100);
}, 100);
// outer fires at ~100ms. When does inner fire? ~200ms? ~100ms?
```

Key Insight / Answer

Trap 1 var: '4, 4, 4' — all closures share same var i (=4 after loop end). let: '1, 2, 3'.

Trap 2: timer fires at ~500ms not 100ms. While loop BLOCKS event loop — timer queued at 100ms but can't run until stack is free.

Trap 3: inner fires at ~200ms (100ms outer + 100ms inner), not 100ms.

Exercise 5 — Build: Async Waterfall using Callbacks

Concept: [Callback Pattern](#) **Task:** Node.js-style `asyncWaterfall(tasks, done)` function likho — tasks ek array of async functions hai. Har function apna result next function ko pass karta hai. Koi bhi error aaye to `done(err)` call karo aur chain rok do:

```
// asyncWaterfall(tasks, done) — implementation
function asyncWaterfall(tasks, done) {
  function runNext(index, result) {
    if (index === tasks.length) {
      done(null, result); // all done!
      return;
    }
    const task = tasks[index];
    try {
      task(result, (err, newResult) => {
        if (err) { done(err); return; } // stop chain
        runNext(index + 1, newResult); // next step
      });
    } catch(e) {
      done(e);
    }
  }
  runNext(0, undefined);
}

// Test it
asyncWaterfall([
  (_, cb) => setTimeout(() => cb(null, 10), 100),
  (n, cb) => setTimeout(() => cb(null, n * 2), 100),
  (n, cb) => setTimeout(() => cb(null, n + 5), 100),
  (n, cb) => {
    if (n > 100) cb(new Error('Too big!'));
    else cb(null, `Result: ${n}`);
  }
], (err, result) => {
  if (err) console.log('Error:', err.message);
  else console.log(result); // 'Result: 25' (10 * 2 + 5)
});
```

Key Insight / Answer

`runNext` recursively calls itself — but async (inside callback), so stack doesn't overflow.

Each task gets (previousResult, callback) — first task gets undefined as result.

Error short-circuits — `done(err)` called immediately, `runNext` never called again.

This is exactly how `async.waterfall` from 'async' npm package works!

Exercise 6 — Build: Debounce + Throttle from Scratch

Concept: [setTimeout Applications](#) **Task:** Debounce aur throttle dono implement karo, phir difference demonstrate karo:

```
// Debounce — waits for silence before firing
function debounce(fn, delay) {
  let timerId = null;
  return function(...args) {
    clearTimeout(timerId);
    timerId = setTimeout(() => {
      fn.apply(this, args);
      timerId = null;
    }, delay);
  };
}

// Throttle — fires at most once per interval
function throttle(fn, limit) {
  let lastCall = 0;
  let timerId = null;
  return function(...args) {
    const now = Date.now();
    const remaining = limit - (now - lastCall);
    if (remaining <= 0) {
      if (timerId) { clearTimeout(timerId); timerId = null; }
      lastCall = now;
      fn.apply(this, args);
    } else if (!timerId) {
      // Schedule trailing call
      timerId = setTimeout(() => {
        lastCall = Date.now();
        timerId = null;
        fn.apply(this, args);
      }, remaining);
    }
  };
}

// Demo: user types fast (every 100ms) for 1 second
const debouncedSearch = debounce(q => console.log('Search:', q), 300);
const throttledLog = throttle(q => console.log('Throttle:', q), 300);

// Simulate typing
['h', 'he', 'hel', 'hell', 'hello'].forEach((v, i) => {
  setTimeout(() => {
    debouncedSearch(v); // fires ONLY ONCE — 300ms after 'hello'
    throttledLog(v); // fires on 'h', then ~300ms later 'hello'
  }, i * 100);
});
```

Key Insight / Answer

Debounce: clears and resets timer on every call — fires only after silence of 300ms.

Throttle: fires immediately, then blocks for 300ms, trailing call scheduled.

Use debounce for: search input, window resize, form auto-save.

Use throttle for: scroll events, mouse move, API rate limiting.